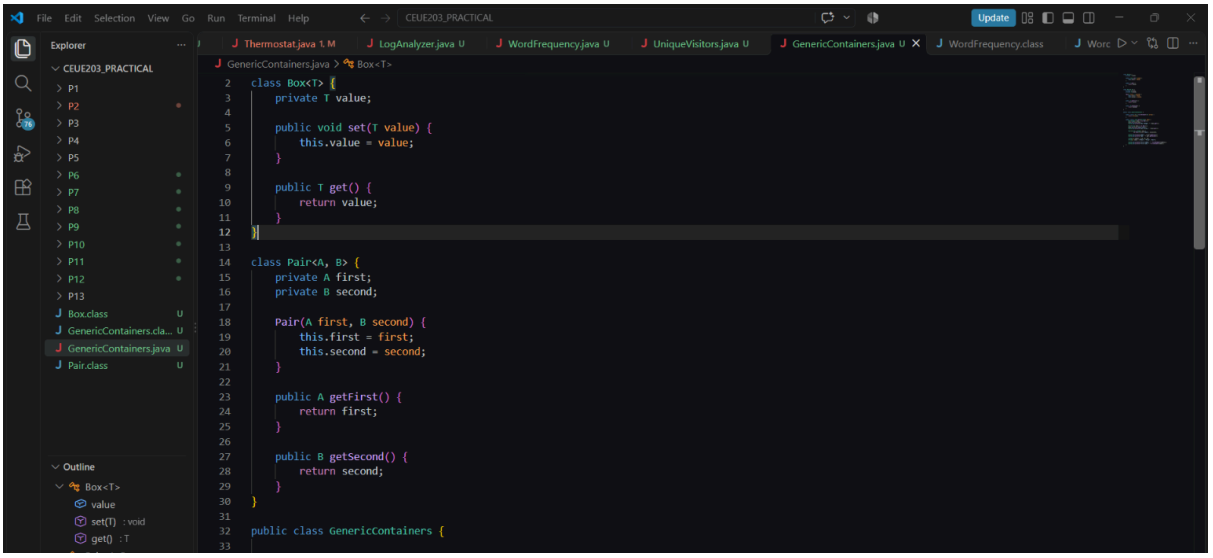
PRACTICAL 13

QUESTION

Generic containers: a generic Box<T> with get/set and a Pair<A,B>; use them with different types to show type safety without casts, and write a generic method that returns the first element of any array.

CODE



```
class Box<T> {
    private T value;

    public void set(T value) {
        this.value = value;
    }

    public T get() {
        return value;
    }
}

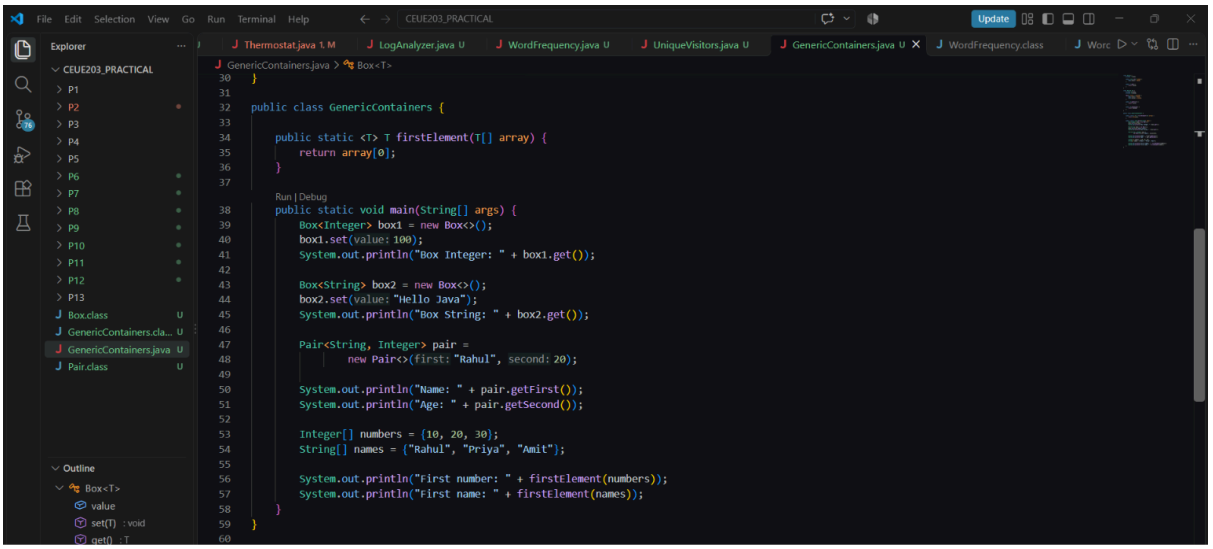
class Pair<A, B> {
    private A first;
    private B second;

    Pair(A first, B second) {
        this.first = first;
        this.second = second;
    }

    public A getFirst() {
        return first;
    }

    public B getSecond() {
        return second;
    }
}

public class GenericContainers {
```



```
}

public class GenericContainers {

    public static <T> T firstElement(T[] array) {
        return array[0];
    }

    Run | Debug
    public static void main(String[] args) {
        Box<Integer> box1 = new Box<>();
        box1.set(value: 100);
        System.out.println("Box Integer: " + box1.get());

        Box<String> box2 = new Box<>();
        box2.set(value: "Hello Java");
        System.out.println("Box String: " + box2.get());

        Pair<String, Integer> pair =
                new Pair<>(first: "Rahul", second: 20);

        System.out.println("Name: " + pair.getFirst());
        System.out.println("Age: " + pair.getSecond());

        Integer[] numbers = {10, 20, 30};
        String[] names = {"Rahul", "Priya", "Amit"};

        System.out.println("First number: " + firstElement(numbers));
        System.out.println("First name: " + firstElement(names));
    }
}
```

OUTPUT

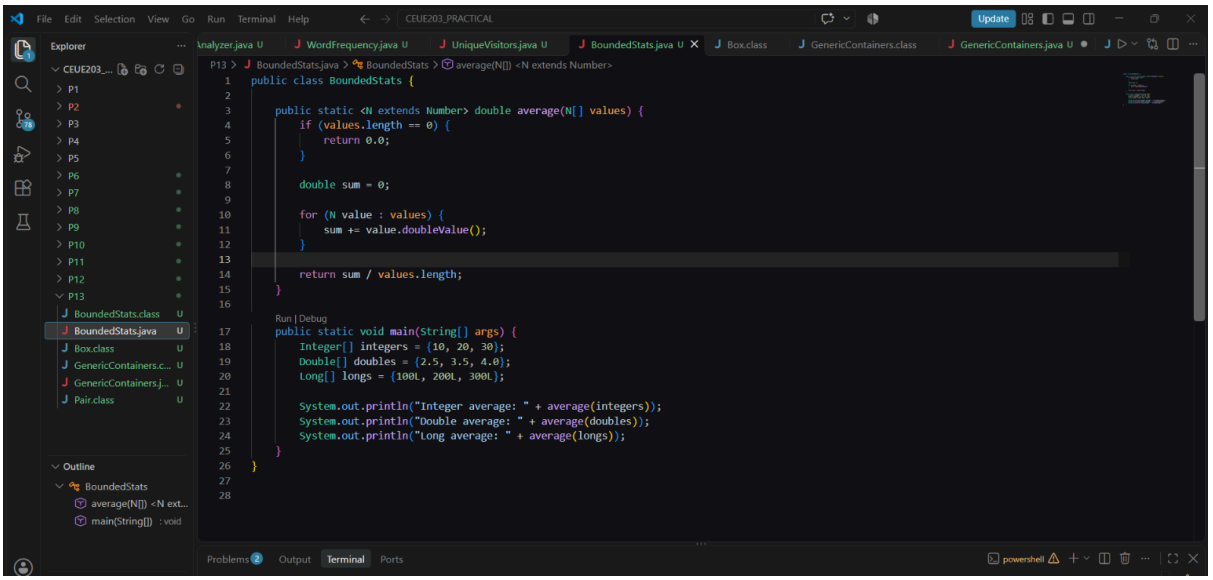


QUESTION

Bounded stats: a generic method <N extends Number> double average(N[] values) that

works for Integer[], Double[] and Long[]; note why it refuses non-Number types.

CODE



OUTPUT



KEY QUESTION

1. **What problem do generics solve compared with using** Object**?**

Generics provide type safety by allowing a class or method to work with specific data types. They reduce the need for type casting and help detect type errors at compile time. Using Object can store different types, but retrieving values may require explicit casting.

2. **What is a bounded type parameter, and why is** <N extends Number> **useful?**

A bounded type parameter restricts the types that can be used as a generic parameter. <N extends Number> allows numeric types such as Integer, Double, and

Long. It is useful for performing numeric operations, such as calculating an average using doubleValue().

3. **What is type erasure?**

Type erasure is the process by which Java removes generic type information during compilation. The compiler replaces type parameters with their bounds or Object where appropriate. It helps maintain compatibility with older Java code.